

Implementation of an Automated Model Inference API Using FastAPI and Docker

CODE:-

```
!pip install fastapi uvicorn pydantic nest-asyncio httpx
```

#API Script File

```
%%writefile app.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Customer Churn Prediction API", version="1.0")

class CustomerData(BaseModel):
    age: int
    tenure: int
    monthly_charges: float

@app.get("/")
def home():
    return {"message": "Welcome to the Churn Prediction API. Use the  
/predict endpoint to get scores."}

@app.post("/predict")
def predict_churn(data: CustomerData):
    # Simulated ML Model Logic
    risk_score = (data.monthly_charges / 120.0) * (1.0 - (data.tenure / 72.0))
    risk_score = min(max(risk_score, 0.0), 1.0)
    prediction = 1 if risk_score > 0.5 else 0

    return {
        "status": "success",
```

```
"input_received": {
    "age": data.age,
    "tenure": data.tenure,
    "monthly_charges": data.monthly_charges
},
"prediction": {
    "churn_probability": round(risk_score, 4),
    "will_churn": prediction
}
}
```

#Start the Server Natively in Colab

```
import nest_asyncio
import uvicorn
from multiprocessing import Process

# Apply the asyncio patch
nest_asyncio.apply()

def run_server():
    uvicorn.run("app:app", host="127.0.0.1", port=8000, log_level="info")

# Run the server in a separate background process so your notebook isn't
# locked up
server_process = Process(target=run_server)
server_process.start()
print("API Server started successfully in the background on port 8000!")
```

#Test the API inside Colab

```
import httpx

# Test the base endpoint
with httpx.Client() as client:
```

```

response = client.get("http://127.0.0.1:8000/")
print("Base Endpoint Response:", response.json())

print("-" * 50)

# Test the /predict endpoint with sample data
sample_payload = {
    "age": 34,
    "tenure": 6,
    "monthly_charges": 95.50
}

with httpx.Client() as client:
    response = client.post("http://127.0.0.1:8000/predict",
json=sample_payload)
    print("Predict Endpoint JSON Response:\n")
    import json
    print(json.dumps(response.json(), indent=2))

```

#Dockerfile for your GitHub deliverable?

```

%%writefile Dockerfile
FROM python:3.10-slim
WORKDIR /app
RUN pip install --no-cache-dir fastapi uvicorn pydantic
COPY app.py .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

JUPYTER NOTEBOOK LINK:-

https://colab.research.google.com/drive/1hcIvveSgmGtFrd472_-vumUZ_aFrLIZl?usp=sharing

OUTPUT SCREENSHOTS:-

```
}  
Writing app.py
```

API Server started successfully in the background on port 8000!

```
--- Root Endpoint Test ---  
{'message': 'Welcome to the Churn Prediction API. Use the /predict endpoint to get scores.'}  
  
-----  
  
--- Predict Endpoint JSON Response ---  
{  
  "status": "success",  
  "input_received": {  
    "age": 34,  
    "tenure": 6,  
    "monthly_charges": 95.5  
  },  
  "prediction": {  
    "churn_probability": 0.7295,  
    "will_churn": 1  
  }  
}
```

Writing Dockerfile

EXPLANATION:-

1. Requirements & Core API (**app.py**)

- Installs **FastAPI** for building the web service and **Uvicorn** to run it.
- Uses **Pydantic** (**BaseModel**) to strictly validate that the incoming customer data types (integers/floats) are correct before processing.
- Exposes two paths: a simple **GET** / health check and a **POST** **/predict** endpoint that computes a simulated churn probability score based on a customer's tenure and charges.

2. Background Execution & Testing

- Uses **nest-asyncio** and Python's **multiprocessing** to run the Uvicorn web server seamlessly inside Google Colab's background without freezing your active notebook cells.
- Uses an HTTP client (**httpx**) to send an internal test request to the running server (**localhost:8000**), verifying that the API successfully returns structural JSON predictions.

3. Containerization (**Dockerfile**)

- Creates a script blueprint to wrap the application inside an isolated **Docker** container using a lightweight Python image (`python:3.10-slim`).
- Configures the container to automatically map network communication to port `8000` and launch the Uvicorn production server upon deployment.